

CO4_AT2_Industry-based Problem

Name: C. Himaja

Reg. No: 19273035

Report

Title

Cloud Resource Allocation Optimization Using 0/1 Knapsack Algorithm

Introduction

Cloud service providers have limited computing resources such as server memory, CPU, storage, and network bandwidth. Multiple enterprise applications compete for these resources, and each application generates a different amount of revenue.

In this problem, the available server memory is limited to 120 GB. Each application requires a specific amount of memory and provides a specific revenue contribution.

The objective is to select applications such that:

- Total allocated memory does not exceed 120 GB.
- Each application is either completely selected or rejected.
- Total revenue is maximized.

This is a 0/1 Knapsack problem because each application can be selected at most once.

Problem Statement

A cloud service provider has a server with 120 GB of available memory.

There are multiple enterprise applications with different memory requirements and revenue contributions.

The problem is to determine which applications should receive memory so that the provider obtains the maximum possible revenue without exceeding 120 GB.

Application Data

Application Memory Required Revenue

App1	30 GB	60
App2	20 GB	50

Application Memory Required Revenue

App3	40 GB	70
App4	10 GB	25
App5	50 GB	80
App6	25 GB	45
App7	15 GB	35
App8	35 GB	65
Capacity	120 GB	

Knapsack Model

Each application is considered an item.

Let:

- W_i = memory requirement of application i
- P_i = revenue/profit generated by application i
- X_i = decision variable

where:

$X_i = 1 \rightarrow$ application is selected

$X_i = 0 \rightarrow$ application is not selected

Objective

Maximize:

Maximize $\sum (P_i \times X_i)$

Subject to:

$\sum (W_i \times X_i) \leq 120 \text{ GB}$

and:

$X_i \in \{0,1\}$

Why 0/1 Knapsack?

The 0/1 Knapsack algorithm is appropriate because an enterprise application cannot normally be partially selected.

For example:

App1 = 30 GB

The provider either allocates the required application resources or does not allocate the application.

Therefore:

0 → Do not allocate App1

1 → Allocate App1

This makes the problem a 0/1 Knapsack problem.

Dynamic Programming Approach

A DP table is constructed using:

$dp[i][w]$

where:

- i = number of applications considered
- w = available memory capacity
- $dp[i][w]$ = maximum revenue obtainable using the first i applications with w GB memory.

Recurrence

If the current application requires more memory than available:

$dp[i][w] = dp[i-1][w]$

Otherwise:

$dp[i][w] =$

$\max($

$dp[i-1][w],$

$\text{profit}[i] + dp[i-1][w - \text{memory}[i]]$

$)$

The first option means reject the application.

The second option means accept the application.

Optimal Substructure

The problem has optimal substructure because an optimal solution to the complete resource allocation problem contains optimal solutions to smaller subproblems.

For example, when deciding whether to select App5, the algorithm considers the best possible allocation for the remaining memory using the previous applications.

Thus:

Optimal Solution

↓

Best solution for smaller capacity

+

Current application's profit

This property allows Dynamic Programming to construct the final optimal solution from smaller solutions.

Overlapping Subproblems

The same subproblems occur multiple times when solving the Knapsack problem recursively.

For example:

`dp[3][50]`

may be required while evaluating several different combinations.

Instead of calculating the same result repeatedly, Dynamic Programming stores the result in a table.

Therefore, previously calculated results can be reused.

This reduces unnecessary computation.

Constraints in Cloud Resource Allocation

1. Service-Level Agreements (SLAs)

Some applications may have strict SLA requirements regarding availability and performance.

A high-priority application may need to be allocated resources even if it provides lower revenue.

2. Workload Fluctuations

Cloud workloads can change over time.

For example:

Morning → Low workload

Afternoon → High workload

Night → Low workload

The resource allocation strategy should therefore adapt to changing workloads.

3. Priority Handling

Applications can have different priorities.

For example:

Critical Application → High priority

Business Application → Medium priority

Non-critical Application → Low priority

The model can be extended by adding priority weights or mandatory-selection constraints.

4. Resource Limits

The total allocated memory must not exceed:

120 GB

5. Application Dependencies

Some applications may require other applications to be running.

For example:

App A → requires App B

Such dependencies can be incorporated into an extended optimization model.

Real-Time Cloud Extension

The basic 0/1 Knapsack algorithm uses fixed application requirements and profits.

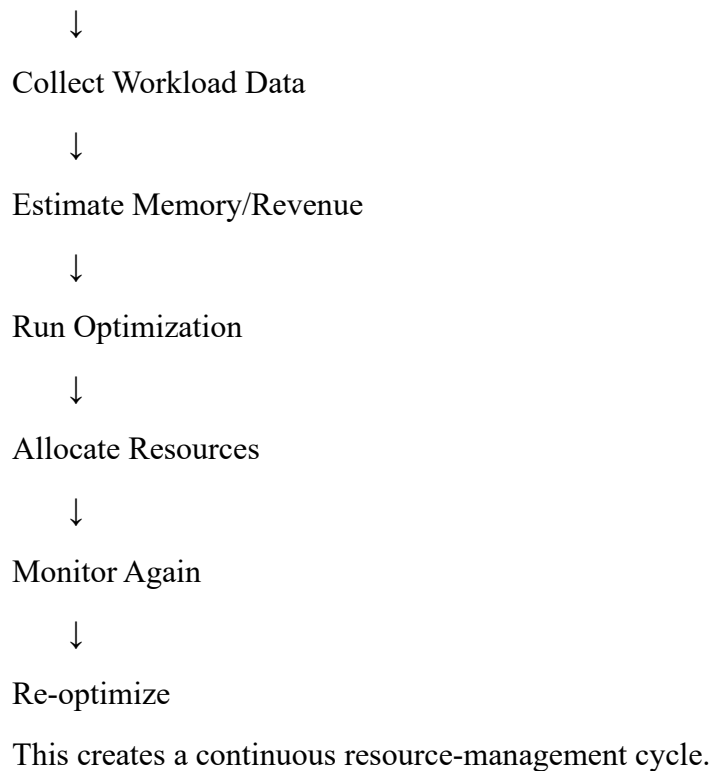
In a real cloud environment, these values may change continuously.

The model can be extended using:

- Real-time monitoring of memory utilization.
- Dynamic workload prediction.
- Auto-scaling.
- Priority-aware allocation.
- SLA-aware resource scheduling.
- Periodic re-optimization.
- Cloud monitoring and telemetry systems.

For example:

Monitor Resources



Scalable Industry Scenario

Consider a cloud provider supporting thousands of enterprise applications.

Instead of only considering memory, the optimization model can consider multiple resources:

Memory

CPU

Storage

Network Bandwidth

GPU

The problem then becomes a multi-resource optimization problem.

For a large-scale cloud environment, the basic DP implementation may become expensive because its complexity depends on the capacity.

Practical systems can therefore combine optimization with:

- Heuristic algorithms
- Approximation algorithms
- Distributed optimization
- Auto-scaling policies

- Priority-based scheduling
- Machine-learning-based workload prediction

Time Complexity

For n applications and capacity W :

Time Complexity = $O(n \times W)$

For this problem:

$n = 8$

$W = 120$

Therefore, the DP table contains approximately:

$8 \times 120 = 960$

states, excluding the extra initialization row and column.

This is efficient for a small number of applications and a moderate memory capacity.

Space Complexity

The program uses a two-dimensional DP table:

$dp[n+1][W+1]$

Therefore:

Space Complexity = $O(n \times W)$

The program also uses an array to store the selected applications.

Performance Analysis

The Dynamic Programming approach is significantly better than checking every possible subset.

For n applications, brute-force enumeration may require:

$O(2^n)$

possible combinations.

Dynamic Programming reduces this to:

$O(n \times W)$

when memory capacity is represented as discrete units.

Thus, DP is suitable for moderate-sized resource allocation problems.

Result

The Cloud Resource Allocation problem was successfully modeled and implemented using the 0/1 Knapsack algorithm and Dynamic Programming in C.

The program considers the memory requirements and revenue contributions of multiple enterprise applications and selects an optimal combination without exceeding the 120 GB server-memory capacity.

For the sample dataset, the program obtains a maximum revenue of:

Maximum Revenue = 240

while using:

Total Memory $\leq$ 120 GB

The approach demonstrates the concepts of optimal substructure, overlapping subproblems, resource constraints, and optimal allocation.

Screenshots to Include Report:

The screenshot shows a C program named 'main.c' in a code editor. The program defines a structure for applications with memory and profit, and implements a 0/1 Knapsack algorithm. The output window displays the following information:

```
=====
CLOUD RESOURCE ALLOCATION OPTIMIZATION
=====

Server Memory Capacity: 120 GB

Application Details:
-----
Application Memory(GB)    Revenue
-----
App1      30              60
App2      20              50
App3      40              70
App4      10              25
App5      50              80
App6      25              45
App7      15              35
App8      35              65
```

The screenshot shows the same C program with the 'knapsack' function implemented. The output window displays the following information:

```
App1      30              60
App2      20              50
App3      40              70
App4      10              25
App5      50              80
App6      25              45
App7      15              35
App8      35              65

Selected Applications:
-----
App1 -> 30 GB -> Revenue 60
App2 -> 20 GB -> Revenue 50
App4 -> 10 GB -> Revenue 25
App6 -> 25 GB -> Revenue 45
App8 -> 35 GB -> Revenue 65
-----

Total Memory Allocated = 120 GB
Unused Memory = 0 GB
Maximum Revenue = 245

=====
=== Code Execution Successful ===
```


main.cRun

```
14 int knapsack(struct Application apps[], int n, int capacity, int
27     for (i = 1; i <= n; i++) {
28         for (w = 0; w <= capacity; w++) {
34             else {
37                 // Excluding app i
38                 int exclude = dp[i - 1][w];
39
40                 // Including app i
41                 int include =
42                     apps[i - 1].profit +
43                     dp[i - 1][w - apps[i - 1].memory];
44
45                 if (include > exclude)
46                     dp[i][w] = include;
47                 else
48                     dp[i][w] = exclude;
49             }
50         }
51     }
52     // Find selected applications by backtracking
```

Output

```
App1  30  60
App2  20  50
App4  10  25
App6  25  45
App8  35  65

Selected Applications:
-----
App1 -> 30 GB -> Revenue 60
App2 -> 20 GB -> Revenue 50
App4 -> 10 GB -> Revenue 25
App6 -> 25 GB -> Revenue 45
App8 -> 35 GB -> Revenue 65
-----

Total Memory Allocated = 120 GB
Unused Memory = 0 GB
Maximum Revenue = 245

=====

=== Code Execution Successful ===
```

main.cRun

```
14 int knapsack(struct Application apps[], int n, int capacity, int
52     // Find selected applications by backtracking
53     w = capacity;
54
55     for (i = n; i > 0; i--) {
56
57         if (dp[i][w] != dp[i - 1][w]) {
58             selected[i - 1] = 1;
59             w = w - apps[i - 1].memory;
60         }
61         else {
62             selected[i - 1] = 0;
63         }
64     }
65
66     return dp[n][capacity];
67 }
68
69 int main() {
70
```

Output

```
App1  30  60
App2  20  50
App4  10  25
App6  25  45
App8  35  65

Selected Applications:
-----
App1 -> 30 GB -> Revenue 60
App2 -> 20 GB -> Revenue 50
App4 -> 10 GB -> Revenue 25
App6 -> 25 GB -> Revenue 45
App8 -> 35 GB -> Revenue 65
-----

Total Memory Allocated = 120 GB
Unused Memory = 0 GB
Maximum Revenue = 245

=====

=== Code Execution Successful ===
```